

Lab 4.1
Client-Server Applications in C++:
From Echo Server to Simple Web Server

Santiago Soto
M00121124
CS 375 Operating Systems

October 7, 2026

Contents

1	Overview	2
2	Part 1: Simple Echo Server	2
2.1	Ten sample requests	2
2.2	Extension: multi-line and long messages	3
2.3	Queuing	4
2.4	Error handling	5
3	Part 2: Multi-Threaded Echo Server	5
3.1	Concurrent requests	5
3.2	Performance compared with Part 1	6
3.3	Shared connection counter	7
3.4	Discussion: thread creation overhead	7
4	Part 3: Thread-Pool Echo Server	7
4.1	Queuing under load	7
4.2	Synchronisation	9
4.3	Graceful shutdown	10
4.4	Resource use compared with Part 2	10
5	Part 4: Simple HTTP Web Server	11
5.1	Request parsing and response	11
5.2	Ten sample requests	12
5.3	Browser	13
5.4	curl	15
6	Cybersecurity Challenge: Path Traversal	20
6.1	The attack	20
6.2	The defence	20
6.3	Test	21
6.4	Why this matters	22
7	Conclusion	22

1 Overview

This lab builds a network server in four steps. Each step keeps the previous one's socket code and changes only how clients are scheduled:

Part	Program	Concurrency model
1	<code>echo_server</code>	One client at a time
2	<code>multi_threaded_server</code>	One thread per client
3	<code>thread_pool_server</code>	Fixed pool of 10 workers and a task queue
4	<code>http_server</code>	The Part 3 pool, serving HTTP instead of echo

Code shared by every program (socket setup, logging, argument validation and error handling) lives in `src/common.hpp`, so each part's source file contains only what is new in that part.

Environment. Ubuntu 22.04.3 LTS on WSL2, g++ 11.4.0, 16 CPU cores. Built with `g++ -std=c++17 -Wall -Wextra -O2 -pthread`.

Requirements common to all parts.

- **Error handling.** Ports are validated (whole number, 1–65535). Failures of `socket`, `bind`, `listen`, `accept` and `connect` are reported with the reason; a port that is already in use and a refused connection each get a specific hint. Partial writes, interrupted system calls and clients that disconnect early are handled without crashing the server.
- **Logging.** Every event is written to the console with a millisecond timestamp. The logger holds a mutex, so lines from different threads never mix inside a line.
- **Simulated work.** The echo servers take an optional `delay_ms` argument. Echoing a few bytes takes microseconds, which makes queuing and concurrency invisible; the delay stands in for real work so the difference between the designs can be measured.

2 Part 1: Simple Echo Server

The server accepts a client, echoes whatever it sends, closes the connection and only then accepts the next client.

2.1 Ten sample requests

The script `scripts/test_echo.sh` sends ten messages and compares each echo with what was sent.

#	Request	Result
1	Plain message	Echoed correctly (25 bytes)
2	Longer sentence	Echoed correctly (74 bytes)
3	Empty string	Handled; 0 bytes sent and received
4	3000 bytes (larger than the 1024-byte buffer)	Echoed correctly in 3 chunks
5	Multi-line message	Echoed correctly (39 bytes)
6	Special characters	Echoed correctly (46 bytes)
7	Unicode (multi-byte characters)	Echoed correctly (48 bytes)
8	Digits	Echoed correctly (21 bytes)
9	Exactly 1024 bytes	Echoed correctly in 1 chunk
10	Final message	Echoed correctly (20 bytes)

```
[2026-10-06 15:15:15.543] Echo server (single threaded) listening on port 8080
[2026-10-06 15:15:21.381] Client #1 connected from 127.0.0.1:40260
[2026-10-06 15:15:21.382] Client #1 received 25 bytes: Message 1: Hello, server!
[2026-10-06 15:15:21.382] Client #1 disconnected (25 bytes echoed)
[2026-10-06 15:15:21.393] Client #2 connected from 127.0.0.1:40266
[2026-10-06 15:15:21.393] Client #2 received 74 bytes: Message 2: a second, slightly longer sentence with spaces and punctuation.
[2026-10-06 15:15:21.393] Client #2 disconnected (74 bytes echoed)
[2026-10-06 15:15:21.402] Client #3 connected from 127.0.0.1:40280
[2026-10-06 15:15:21.402] Client #3 disconnected (0 bytes echoed)
[2026-10-06 15:15:21.413] Client #4 connected from 127.0.0.1:40284
[2026-10-06 15:15:21.413] Client #4 received 1024 bytes: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
[2026-10-06 15:15:21.413] Client #4 received 1024 bytes: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
[2026-10-06 15:15:21.413] Client #4 received 952 bytes: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA...
[2026-10-06 15:15:21.413] Client #4 disconnected (3080 bytes echoed)
[2026-10-06 15:15:21.425] Client #5 connected from 127.0.0.1:40286
[2026-10-06 15:15:21.425] Client #5 received 39 bytes: Message 5: line one\nline two\nline three
[2026-10-06 15:15:21.426] Client #5 disconnected (39 bytes echoed)
[2026-10-06 15:15:21.440] Client #6 connected from 127.0.0.1:40292
[2026-10-06 15:15:21.440] Client #6 received 46 bytes: Message 6: special chars !@#$%^&*()[]{}<>?/\|~
[2026-10-06 15:15:21.440] Client #6 disconnected (46 bytes echoed)
[2026-10-06 15:15:21.449] Client #7 connected from 127.0.0.1:40304
[2026-10-06 15:15:21.449] Client #7 received 48 bytes: Message 7: unicode - café, naïve, 你好, 🍕
[2026-10-06 15:15:21.449] Client #7 disconnected (48 bytes echoed)
[2026-10-06 15:15:21.458] Client #8 connected from 127.0.0.1:40314
[2026-10-06 15:15:21.458] Client #8 received 21 bytes: Message 8: 1234567890
[2026-10-06 15:15:21.458] Client #8 disconnected (21 bytes echoed)
[2026-10-06 15:15:21.468] Client #9 connected from 127.0.0.1:40320
[2026-10-06 15:15:21.469] Client #9 received 1024 bytes: BBBB8888BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB...
[2026-10-06 15:15:21.469] Client #9 disconnected (1024 bytes echoed)
[2026-10-06 15:15:21.481] Client #10 connected from 127.0.0.1:40324
[2026-10-06 15:15:21.481] Client #10 received 20 bytes: Message 10: goodbye!
[2026-10-06 15:15:21.481] Client #10 disconnected (20 bytes echoed)
```

Figure 1: Part 1, server (`./bin/echo_server 8080`): log of the ten sample requests.

[illegible]

Figure 2: Part 1, client (`scripts/test_echo.sh 8080`): each echo matches what was sent.

```
[2026-10-06 15:17:47.069] Client #11 received 44 bytes: Hello my name is Santiago. This is my lab.\r\n
```

Figure 3: Part 1, server: the line received from the telnet session.

```
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^['.
Hello my name is Santiago. This is my lab.
Hello my name is Santiago. This is my lab.
[]
```

Figure 4: Part 1, client (`telnet localhost 8080`): the typed line is echoed back.

2.2 Extension: multi-line and long messages

The starting code called `read` once with a 1024-byte buffer. TCP is a byte stream, not a sequence of messages: one `read` returns whatever has arrived so far, so anything beyond the

first 1024 bytes was never echoed. The server now reads in a loop and echoes each chunk until `read` returns 0 (end of file). The server log for request 4 shows the 3000-byte message arriving as several chunks.

The client marks the end of its message by half-closing the connection with `shutdown(fd, SHUT_WR)`. That delivers end-of-file to the server while leaving the other direction open for the echo.

The starting client also converted the host with `inet_pton`, which accepts only numeric addresses, so `./client localhost 8080 ...` could not work as written. The client uses `getaddrinfo` instead, which resolves names as well as addresses.

2.3 Queuing

With a 2000 ms delay per client, three clients were started at the same moment.

```
[2026-10-06 15:19:17.401] Echo server (single-threaded) listening on port 8080, delay 2000 ms per client
[2026-10-06 15:19:29.629] Client #1 connected from 127.0.0.1:51804
[2026-10-06 15:19:31.629] Client #1 received 19 bytes: Client 2 says hello
[2026-10-06 15:19:31.630] Client #1 disconnected (19 bytes echoed)
[2026-10-06 15:19:31.630] Client #2 connected from 127.0.0.1:51798
[2026-10-06 15:19:33.630] Client #2 received 19 bytes: Client 1 says hello
[2026-10-06 15:19:33.630] Client #2 disconnected (19 bytes echoed)
[2026-10-06 15:19:33.630] Client #3 connected from 127.0.0.1:51806
[2026-10-06 15:19:35.630] Client #3 received 610 bytes: Client 3: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:19:35.630] Client #3 disconnected (610 bytes echoed)
```

Figure 5: Part 1, server (`./bin/echo_server 8080 2000`): three simultaneous clients are served one after another, two seconds apart.

```
root@Saint-James:~/Lab-4.1-Web-Server# scripts/run_clients.sh 8080 3
Launching 3 clients in parallel against port 8080 [15:28:01]
[client 1] Echo: Client 1 says hello
[client 1]      (sent 19 bytes, received 19 bytes, match)
[client 2] Echo: Client 2 says hello
[client 2]      (sent 19 bytes, received 19 bytes, match)
[client 3] Echo: Client 3: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx... [truncated on screen]
[client 3]      (sent 610 bytes, received 610 bytes, match)

All 3 clients finished in 6.009 seconds (0 failed)
```

Figure 6: Part 1, client (`scripts/run_clients.sh 8080 3`): total time for the three clients.

All three connections are established immediately, because the kernel completes the TCP handshake and holds finished connections in the listen backlog. The server, however, is busy with the first client and does not call `accept` again until it is done. The timestamps show the clients being served about two seconds apart, for a total of 6.0 s. One slow client therefore delays everyone behind it.

2.4 Error handling

```

❶ root@Saint-James:~/Lab-4.1-Web-Server# ./bin/echo_server abc
Error: Invalid port 'abc' (must be a number from 1 to 65535)
Usage: ./bin/echo_server [port] [delay_ms]
    port      1-65535 (default 8080)
    delay_ms  0-60000, simulated work per client (default 0)
❷ root@Saint-James:~/Lab-4.1-Web-Server# ./bin/echo_server 99999
Error: Invalid port '99999' (must be a number from 1 to 65535)
Usage: ./bin/echo_server [port] [delay_ms]
    port      1-65535 (default 8080)
    delay_ms  0-60000, simulated work per client (default 0)
❸ root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 7: Server started with an invalid port (`abc`) and an out-of-range port (`99999`).

```
connect to localhost:8080 failed: Connection refused (is the server running?)
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 8: Client run while no server is listening: connection refused.

3 Part 2: Multi-Threaded Echo Server

The main thread now only accepts connections. Each client is handed to a new detached `pthread`, so a slow client no longer blocks the others.

3.1 Concurrent requests

```
root@Saint-James:~/Lab-4.1-Web-Server# ./bin/multi_threaded_server 8080 500
[2026-10-06 15:36:56.507] Multi-threaded echo server listening on port 8080, delay 500 ms per client
[2026-10-06 15:37:17.787] Client #1 connected from 127.0.0.1:33862 (total connections: 1, active threads: 1)
[2026-10-06 15:37:17.788] Client #2 connected from 127.0.0.1:33860 (total connections: 2, active threads: 2)
[2026-10-06 15:37:17.791] Client #3 connected from 127.0.0.1:33876 (total connections: 3, active threads: 3)
[2026-10-06 15:37:17.791] Client #4 connected from 127.0.0.1:33890 (total connections: 4, active threads: 4)
[2026-10-06 15:37:17.792] Client #5 connected from 127.0.0.1:33906 (total connections: 5, active threads: 5)
[2026-10-06 15:37:17.795] Client #6 connected from 127.0.0.1:33920 (total connections: 6, active threads: 6)
[2026-10-06 15:37:17.796] Client #7 connected from 127.0.0.1:33928 (total connections: 7, active threads: 7)
[2026-10-06 15:37:17.796] Client #8 connected from 127.0.0.1:33944 (total connections: 8, active threads: 8)
[2026-10-06 15:37:17.811] Client #9 connected from 127.0.0.1:33954 (total connections: 9, active threads: 9)
[2026-10-06 15:37:17.811] Client #10 connected from 127.0.0.1:33962 (total connections: 10, active threads: 10)
[2026-10-06 15:37:18.288] Client #1 received 19 bytes: Client 1 says hello
[2026-10-06 15:37:18.288] Client #2 received 19 bytes: Client 2 says hello
[2026-10-06 15:37:18.288] Client #1 disconnected (19 bytes echoed, 9 threads still active)
[2026-10-06 15:37:18.288] Client #2 disconnected (19 bytes echoed, 8 threads still active)
[2026-10-06 15:37:18.292] Client #3 received 610 bytes: Client 3: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:37:18.292] Client #4 received 19 bytes: Client 4 says hello
[2026-10-06 15:37:18.292] Client #4 disconnected (19 bytes echoed, 7 threads still active)
[2026-10-06 15:37:18.292] Client #5 received 19 bytes: Client 5 says hello
[2026-10-06 15:37:18.293] Client #3 disconnected (610 bytes echoed, 6 threads still active)
[2026-10-06 15:37:18.293] Client #5 disconnected (19 bytes echoed, 5 threads still active)
[2026-10-06 15:37:18.296] Client #6 received 19 bytes: Client 7 says hello
[2026-10-06 15:37:18.296] Client #7 received 19 bytes: Client 8 says hello
[2026-10-06 15:37:18.296] Client #6 disconnected (19 bytes echoed, 4 threads still active)
[2026-10-06 15:37:18.296] Client #8 received 1024 bytes: Client 6: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:37:18.296] Client #8 received 186 bytes: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:37:18.296] Client #7 disconnected (19 bytes echoed, 3 threads still active)
[2026-10-06 15:37:18.296] Client #8 disconnected (1210 bytes echoed, 2 threads still active)
[2026-10-06 15:37:18.311] Client #9 received 20 bytes: Client 10 says hello
[2026-10-06 15:37:18.312] Client #9 disconnected (20 bytes echoed, 1 threads still active)
[2026-10-06 15:37:18.314] Client #10 received 1024 bytes: Client 9: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:37:18.314] Client #10 received 786 bytes: xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx...
[2026-10-06 15:37:18.315] Client #10 disconnected (1810 bytes echoed, 0 threads still active)
```

Figure 9: Part 2, server: ten parallel clients. The log lines of different clients interleave.

In Part 1 every client’s “connected / received / disconnected” lines appeared as one block. Here all ten clients connect within a few milliseconds and their lines are interleaved, which shows that they are being served at the same time.

3.2 Performance compared with Part 1

Ten clients, 500 ms of simulated work per client:

Server	Clients	Total time
Part 1, single-threaded	10	5.02 s
Part 2, thread per client	10	0.55 s

[illegible]

Figure 10: Part 1 under the timing test.


```

[2026-10-06 16:17:52.983] Client #3 queued from 127.0.0.1:45464 (queue size: 1)
[2026-10-06 16:17:52.983] Worker 3 / Client #3 dequeued (queue size: 0)
[2026-10-06 16:17:52.983] Client #4 queued from 127.0.0.1:45480 (queue size: 1)
[2026-10-06 16:17:52.983] Client #5 queued from 127.0.0.1:45492 (queue size: 1)
[2026-10-06 16:17:52.983] Worker 4 / Client #4 dequeued (queue size: 0)
[2026-10-06 16:17:52.984] Worker 5 / Client #5 dequeued (queue size: 0)
[2026-10-06 16:17:52.988] Client #6 queued from 127.0.0.1:45494 (queue size: 1)
[2026-10-06 16:17:52.989] Worker 6 / Client #6 dequeued (queue size: 0)
[2026-10-06 16:17:52.989] Client #7 queued from 127.0.0.1:45496 (queue size: 1)
[2026-10-06 16:17:52.989] Worker 7 / Client #7 dequeued (queue size: 0)
[2026-10-06 16:17:52.910] Client #8 queued from 127.0.0.1:45510 (queue size: 1)
[2026-10-06 16:17:52.910] Worker 8 / Client #8 dequeued (queue size: 0)
[2026-10-06 16:17:52.917] Client #9 queued from 127.0.0.1:45522 (queue size: 1)
[2026-10-06 16:17:52.917] Client #10 queued from 127.0.0.1:45536 (queue size: 2)
[2026-10-06 16:17:52.917] Worker 9 / Client #9 dequeued (queue size: 1)
[2026-10-06 16:17:52.917] Worker 10 / Client #10 dequeued (queue size: 0)
[2026-10-06 16:17:52.919] Client #11 queued from 127.0.0.1:45552 (queue size: 1)
[2026-10-06 16:17:52.924] Client #12 queued from 127.0.0.1:45566 (queue size: 2)
[2026-10-06 16:17:52.925] Client #13 queued from 127.0.0.1:45572 (queue size: 3)
[2026-10-06 16:17:52.926] Client #14 queued from 127.0.0.1:45578 (queue size: 4)
[2026-10-06 16:17:52.928] Client #15 queued from 127.0.0.1:45586 (queue size: 5)
[2026-10-06 16:17:52.929] Client #16 queued from 127.0.0.1:45592 (queue size: 6)
[2026-10-06 16:17:52.929] Client #17 queued from 127.0.0.1:45598 (queue size: 7)
[2026-10-06 16:17:52.937] Client #18 queued from 127.0.0.1:45614 (queue size: 8)
[2026-10-06 16:17:52.937] Client #19 queued from 127.0.0.1:45622 (queue size: 9)
[2026-10-06 16:17:52.937] Client #20 queued from 127.0.0.1:45628 (queue size: 10)
[2026-10-06 16:17:53.402] Worker 2 / Client #11 dequeued (queue size: 9)
[2026-10-06 16:17:53.402] Worker 1 / Client #12 dequeued (queue size: 8)
[2026-10-06 16:17:53.404] Worker 3 / Client #13 dequeued (queue size: 7)
[2026-10-06 16:17:53.404] Worker 4 / Client #14 dequeued (queue size: 6)
[2026-10-06 16:17:53.404] Worker 5 / Client #15 dequeued (queue size: 5)
[2026-10-06 16:17:53.409] Worker 7 / Client #16 dequeued (queue size: 4)
[2026-10-06 16:17:53.409] Worker 6 / Client #17 dequeued (queue size: 3)
[2026-10-06 16:17:53.411] Worker 8 / Client #18 dequeued (queue size: 2)
[2026-10-06 16:17:53.418] Worker 10 / Client #19 dequeued (queue size: 1)
[2026-10-06 16:17:53.418] Worker 9 / Client #20 dequeued (queue size: 0)

```

Figure 12: Part 3, server, queue lines only: clients 11–20 are queued while all ten workers are busy, then dequeued as workers become free.

```

[2026-10-06 15:50:26.802] Worker 9 / Client #9 received 1024 bytes: Client 9: .....
[2026-10-06 15:50:26.802] Worker 10 / Client #10 done (20 bytes echoed)
[2026-10-06 15:50:26.803] Worker 10 / Client #19 dequeued (queue size: 1)
[2026-10-06 15:50:26.803] Worker 9 / Client #9 received 786 bytes: .....
[2026-10-06 15:50:26.803] Worker 9 / Client #9 done (1810 bytes echoed)
[2026-10-06 15:50:26.803] Worker 9 / Client #20 dequeued (queue size: 0)
[2026-10-06 15:50:27.272] Worker 2 / Client #11 received 20 bytes: Client 11 says hello
[2026-10-06 15:50:27.272] Worker 2 / Client #11 done (20 bytes echoed)
[2026-10-06 15:50:27.273] Worker 1 / Client #12 received 1024 bytes: Client 12: .....
[2026-10-06 15:50:27.273] Worker 1 / Client #12 received 1024 bytes: .....
[2026-10-06 15:50:27.273] Worker 1 / Client #12 received 363 bytes: .....
[2026-10-06 15:50:27.273] Worker 1 / Client #12 done (2411 bytes echoed)
[2026-10-06 15:50:27.275] Worker 3 / Client #13 received 20 bytes: Client 14 says hello
[2026-10-06 15:50:27.275] Worker 3 / Client #13 done (20 bytes echoed)
[2026-10-06 15:50:27.276] Worker 4 / Client #14 received 20 bytes: Client 13 says hello
[2026-10-06 15:50:27.276] Worker 4 / Client #14 done (20 bytes echoed)
[2026-10-06 15:50:27.277] Worker 5 / Client #15 received 1024 bytes: Client 15: .....
[2026-10-06 15:50:27.278] Worker 5 / Client #15 received 1024 bytes: .....
[2026-10-06 15:50:27.278] Worker 5 / Client #15 received 963 bytes: .....
[2026-10-06 15:50:27.278] Worker 5 / Client #15 done (3011 bytes echoed)
[2026-10-06 15:50:27.286] Worker 6 / Client #16 received 20 bytes: Client 16 says hello
[2026-10-06 15:50:27.286] Worker 6 / Client #16 done (20 bytes echoed)
[2026-10-06 15:50:27.287] Worker 7 / Client #17 received 20 bytes: Client 17 says hello
[2026-10-06 15:50:27.287] Worker 7 / Client #17 done (20 bytes echoed)
[2026-10-06 15:50:27.290] Worker 8 / Client #18 received 1024 bytes: Client 18: .....
[2026-10-06 15:50:27.290] Worker 8 / Client #18 received 1024 bytes: .....
[2026-10-06 15:50:27.290] Worker 8 / Client #18 received 539 bytes: .....
[2026-10-06 15:50:27.290] Worker 8 / Client #18 done (3611 bytes echoed)
[2026-10-06 15:50:27.303] Worker 10 / Client #19 received 20 bytes: Client 19 says hello
[2026-10-06 15:50:27.303] Worker 9 / Client #20 received 20 bytes: Client 20 says hello
[2026-10-06 15:50:27.303] Worker 10 / Client #19 done (20 bytes echoed)
[2026-10-06 15:50:27.303] Worker 9 / Client #20 done (20 bytes echoed)

```

Figure 13: Part 3, server, full log: the last clients leave the queue and the second batch is served by the same ten workers about 500 ms after the first.

4.3 Graceful shutdown

[illegible]

Figure 16: Part 3, server: Ctrl+C while clients are still being served. Work already accepted is finished before the server exits.

A SIGINT/SIGTERM handler only sets a flag. The handler is installed without `SA_RESTART`, so the blocked `accept` returns `EINTR` and the main loop sees the flag. The signals are blocked while the workers are created, which guarantees delivery to the main thread. The server then stops accepting, sets `stop_pool`, wakes every worker and joins them. A worker exits only when the pool is stopping *and* the queue is empty, so every client that was already accepted is still served; the screenshot shows the queue being drained before “Shutdown complete”.

4.4 Resource use compared with Part 2

Both servers were given twenty clients with a 15 000 ms delay and inspected with `ps` while the clients were connected.

Server	Clients	Threads (NLWP)	Memory (RSS)	CPU
Part 2, thread per client	20	21	4348 kB	0.0 %
Part 3, pool of 10	20	11	4068 kB	0.0 %

```
root@Saint-James:~/Lab-4.1-Web-Server# ps -o pid,nlwp,rss,pcpu,cmd -p "$(pgrep -f bin/multi_threaded_server)"
  PID NLWP   RSS %CPU CMD
  89972   21 4348  0.0 ./bin/multi_threaded_server 8080 15000
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 17: Part 2 under load: 21 threads, one per connected client plus the main thread.

```

● root@Saint-James:~/Lab-4.1-Web-Server# ps -o pid,nlwp,rss,pcpu,cmd -p "$(pgrep -f bin/thread_pool_server)"
  PID NLWP  RSS %CPU CMD
  88927   11 4068  0.0 ./bin/thread_pool_server 8080 15000
○ root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 18: Part 3 under load: 11 threads, the same as when idle.

Part 2's thread count follows the number of clients, while Part 3 stays at eleven (ten workers and the main thread) regardless of load. CPU use is zero in both because the threads are asleep in the simulated delay, and resident memory differs by only 280 kB: a thread's stack is reserved address space that costs real memory only once it is used, and these threads do very little. At twenty clients the difference is therefore small. What matters is how it grows: Part 2 adds a thread for every further client without limit, and Part 3 adds none. The price is latency: when all workers are busy, a new client waits in the queue. That is the better failure mode, because an overloaded pool becomes slower while an overloaded thread-per-client server runs out of resources.

5 Part 4: Simple HTTP Web Server

The Part 3 pool is reused unchanged; the workers now parse an HTTP request and send a file from `./www`.

5.1 Request parsing and response

`parse_request` reads until the blank line that ends the headers (a request may arrive in several pieces), then:

- checks that the request line is exactly *method*, *target*, *version*, with an HTTP/1.x version and a target starting with `/`;
- parses the headers into a map with lower-cased names;
- splits the target into path and query string, percent-decodes the path (`\%20` becomes a space) and parses the query parameters;
- reads a POST body of exactly `Content-Length` bytes.

GET and HEAD serve files, POST echoes the body it received, and any other method is answered with 405. Malformed requests receive 400, oversized headers 431, oversized bodies 413 and clients that stall 408. A five-second socket timeout stops an idle connection from occupying a worker indefinitely. The full request (request line, headers, query parameters and body) is written to the log.

Files are read in binary mode so that images are not altered, and the response carries the exact `Content-Length`. The `Content-Type` is chosen from the file extension. This matters in practice: a browser will not apply a stylesheet that is not served as `text/css`, and needs `image/png` to render the logo.

5.2 Ten sample requests

#	Request	Expected	Status	Content-Type
1	/	200	200	text/html
2	/index.html	200	200	text/html
3	/style.css	200	200	text/css
4	/logo.png	200	200	image/png
5	/subpage.html	200	200	text/html
6	/missing.txt	404	404	text/html (error page)
7	/favicon.ico	404	404	text/html (error page)
8	/..	404	404	text/html (error page)
9	/?test=1	200	200	text/html
10	/very/long/path/to/file.html	404	404	text/html (error page)

```

root@Saint-James:~/Lab-4.1-Web-Server# scripts/test_http.sh 8080
=== 10 sample requests [2026-10-07 17:37:01] ===
/                200 text/html; charset=utf-8    499 B ok  serves index.html
/index.html      200 text/html; charset=utf-8    499 B ok
/style.css       200 text/css; charset=utf-8      88 B ok  text/css
/logo.png       200 image/png                    5528 B ok image/png
/subpage.html    200 text/html; charset=utf-8    134 B ok
/missing.txt     404 text/html; charset=utf-8    141 B ok  does not exist
/favicon.ico     404 text/html; charset=utf-8    141 B ok  does not exist
/..              404 text/html; charset=utf-8    141 B ok  security test
/?test=1        200 text/html; charset=utf-8    499 B ok  query string
/very/long/path/to/file.html 404 text/html; charset=utf-8    141 B ok  does not exist

=== Path traversal attacks (all must be 404) ===
/../../../../etc/passwd 404 text/html; charset=utf-8    141 B ok  plain ../
/..%2e%2e/%2e%2e/etc/passwd 404 text/html; charset=utf-8    141 B ok  encoded dots (from the lab sheet)
/%2e%2e/%2e%2e/%2e%2e/etc/passwd 404 text/html; charset=utf-8    141 B ok  encoded dots
/..%2f..%2f..%2fetc%2fpasswd 404 text/html; charset=utf-8    141 B ok  encoded slashes
/subdir/../../../../etc/passwd 404 text/html; charset=utf-8    141 B ok  starts inside the root
/%00             400 text/html; charset=utf-8    145 B ok  NUL byte
/%zz             400 text/html; charset=utf-8    145 B ok  malformed escape

All checks passed.
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 19: Part 4: all sample requests and traversal attacks, with status, type and size.

5.3 Browser



Figure 20: The index page with its stylesheet and logo applied.

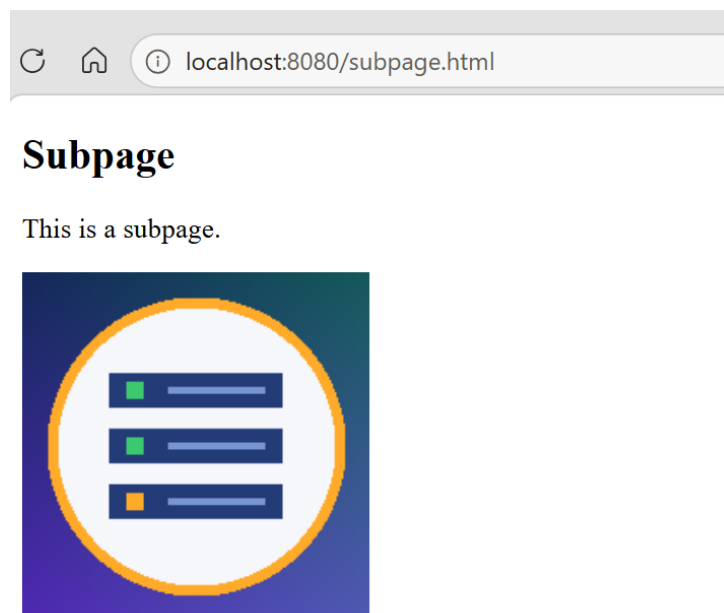
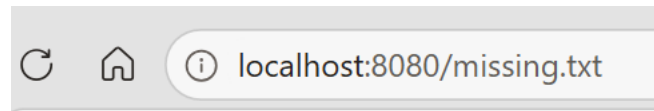


Figure 21: The subpage.



404 Not Found

Lab 4.1 C++ web server

Figure 22: The 404 page for the missing link.

```
| sec-ch-ua: "Chromium";v="154", "Microsoft Edge";v="154", "Not A(Brand";v="99"
| sec-ch-ua-mobile: ?0
| Accept: text/css,*/*;q=0.1
| Sec-Fetch-Site: same-origin
| Sec-Fetch-Mode: no-cors
| Sec-Fetch-Dest: style
| Referer: http://localhost:8080/
| Accept-Encoding: gzip, deflate, br, zstd
| Accept-Language: en-US,en;q=0.9
| Cookie: ajs_anonymous_id=416549ad-0da5-4ef0-9dd4-01ddf3d6821b
[2026-10-07 17:34:16.739] #7 127.0.0.1:38596 "GET /logo.png HTTP/1.1" -> 200 OK (5528 bytes, image/png) [worker 7]
| GET /logo.png HTTP/1.1
| Host: localhost:8080
| Connection: keep-alive
| sec-ch-ua-platform: "Windows"
| User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
| sec-ch-ua: "Chromium";v="154", "Microsoft Edge";v="154", "Not A(Brand";v="99"
| sec-ch-ua-mobile: ?0
| Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
| Sec-Fetch-Site: same-origin
| Sec-Fetch-Mode: no-cors
| Sec-Fetch-Dest: image
| Referer: http://localhost:8080/
| Accept-Encoding: gzip, deflate, br, zstd
| Accept-Language: en-US,en;q=0.9
| Cookie: ajs_anonymous_id=416549ad-0da5-4ef0-9dd4-01ddf3d6821b
[2026-10-07 17:34:16.793] #8 127.0.0.1:38608 "GET /favicon.ico HTTP/1.1" -> 404 Not Found (141 bytes, text/html; charset=utf-8) [worker 8]
| GET /favicon.ico HTTP/1.1
| Host: localhost:8080
| Connection: keep-alive
| sec-ch-ua-platform: "Windows"
| User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
| sec-ch-ua: "Chromium";v="154", "Microsoft Edge";v="154", "Not A(Brand";v="99"
| sec-ch-ua-mobile: ?0
| Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8
| Sec-Fetch-Site: same-origin
| Sec-Fetch-Mode: no-cors
| Sec-Fetch-Dest: image
| Referer: http://localhost:8080/
| Accept-Encoding: gzip, deflate, br, zstd
| Accept-Language: en-US,en;q=0.9
| Cookie: ajs_anonymous_id=416549ad-0da5-4ef0-9dd4-01ddf3d6821b
```

Figure 23: Server log while the browser loads the index page: full requests are logged and handled by several workers.

Loading the index page makes the browser send several requests at once (the page, the stylesheet, the logo and a favicon), which the server log shows being handled by different workers.

5.4 curl

```
root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 499
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Welcome to the Test Server!</h1>
  <p>This is a simple HTML page served by your C++ server.</p>
  
  <ul>
    <li>Link to <a href="/subpage.html">subpage</a></li>
    <li>Non-existent link: <a href="/missing.txt">missing</a> (should 404)</li>
  </ul>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 24: `curl -v /`: request and response headers, then the HTML.

```
root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/style.css
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /style.css HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: text/css; charset=utf-8
< Content-Length: 88
< X-Content-Type-Options: nosniff
< Connection: close
<
body { font-family: Arial, sans-serif; background-color: #f0f0f0; }
h1 { color: #333; }
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 25: `curl -v /style.css`: served as `text/css`.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v -o /dev/null http://localhost:8080/logo.png
* Trying 127.0.0.1:8080...
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0     0  0 --:--:-- --:--:-- --:--:--    0* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /logo.png HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: image/png
< Content-Length: 5528
< X-Content-Type-Options: nosniff
< Connection: close
<
{ [5528 bytes data]
100 5528 100 5528 0 0 3378k 0 --:--:-- --:--:-- --:--:-- 5398k
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 26: `curl -v /logo.png`: served as image/png.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/missing.txt
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /missing.txt HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
<h1>404 Not Found</h1>
<p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 27: `curl -v /missing.txt`: 404 Not Found.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/index.html
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /index.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 499
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Welcome to the Test Server!</h1>
  <p>This is a simple HTML page served by your C++ server.</p>
  
  <ul>
    <li>Link to <a href="/subpage.html">subpage</a></li>
    <li>Non-existent link: <a href="/missing.txt">missing</a> (should 404)</li>
  </ul>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 28: curl -v /index.html.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/subpage.html
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /subpage.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 134
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<body>
  <h2>Subpage</h2>
  <p>This is a subpage.</p>
  
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 29: curl -v /subpage.html.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/favicon.ico
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /favicon.ico HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
<h1>404 Not Found</h1>
<p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 30: curl -v /favicon.ico: 404 Not Found.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v --path-as-is http://localhost:8080/..
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /.. HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
<h1>404 Not Found</h1>
<p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 31: curl -v --path-as-is /..: 404 Not Found.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v "http://localhost:8080/?test=1"
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /?test=1 HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 499
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Welcome to the Test Server!</h1>
  <p>This is a simple HTML page served by your C++ server.</p>
  
  <ul>
    <li>Link to <a href="/subpage.html">subpage</a></li>
    <li>Non-existent link: <a href="/missing.txt">missing</a> (should 404)</li>
  </ul>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 32: `curl -v /?test=1`: the query string is parsed and the index page is served.

```

root@Saint-James:~/Lab-4.1-Web-Server# curl -v http://localhost:8080/very/long/path/to/file.html
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET /very/long/path/to/file.html HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
  <h1>404 Not Found</h1>
  <p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#

```

Figure 33: `curl -v /very/long/path/to/file.html`: 404 Not Found.

6 Cybersecurity Challenge: Path Traversal

6.1 The attack

The server builds a file name by appending the URL path to the web root. A path containing `../` walks back out of that directory: `./www/../../etc/passwd` is `/etc/passwd`. The starting code rejected any path containing `..`, but it ran that check on the raw URL. In `../../\%2e\%2e\%2e\%2e/etc/passwd` the dots are written as `\%2e`, so a check for the two characters `..` finds nothing in the encoded segments, and they turn into `..` only when the path is decoded later. More generally, searching for known-bad strings is fragile: it has to anticipate every encoding, and it cannot detect a symbolic link inside the web root that points outside it, which involves no `..` at all.

6.2 The defence

Instead of looking for bad input, the server works out where the request really leads and checks the result:

1. **Decode once, first.** The path is percent-decoded before any check, so every later step sees what the file system will see. Malformed escapes and `\%00` are rejected with 400.
2. **Canonicalise.** `std::filesystem::weakly_canonical` turns the path into an absolute one with `.`, `..` and symbolic links resolved.
3. **Verify containment.** The resolved path must lie inside the canonical web root, compared component by component. A plain string prefix test would be wrong here: it would accept the directory `/srv/www-secret` as being inside `/srv/www`.

```
fs::path resolved = fs::weakly_canonical(
    g_web_root / fs::path(url_path).relative_path(), ec);
if (ec) return std::nullopt;

auto mismatch = std::mismatch(g_web_root.begin(), g_web_root.end(),
                              resolved.begin(), resolved.end());
if (mismatch.first != g_web_root.end()) {
    lab::log("SECURITY: blocked path traversal attempt: ...");
    return std::nullopt; // answered with 404
}
```

6.3 Test

```
root@Saint-James:~/Lab-4.1-Web-Server# curl -v --path-as-is http://localhost:8080/../../../../etc/passwd
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET ../../../../etc/passwd HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
<h1>404 Not Found</h1>
<p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 34: Traversal attempt with plain ../ segments: 404 Not Found.

```
root@Saint-James:~/Lab-4.1-Web-Server# curl -v --path-as-is "http://localhost:8080/../../../../%2e%2e/%2e%2e/etc/passwd"
* Trying 127.0.0.1:8080...
* Connected to localhost (127.0.0.1) port 8080 (#0)
> GET ../../../../%2e%2e/%2e%2e/etc/passwd HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.81.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 404 Not Found
< Server: Lab4.1-http_server
< Content-Type: text/html; charset=utf-8
< Content-Length: 141
< X-Content-Type-Options: nosniff
< Connection: close
<
<!DOCTYPE html>
<html>
<head><title>404 Not Found</title></head>
<body>
<h1>404 Not Found</h1>
<p>Lab 4.1 C++ web server</p>
</body>
</html>
* Closing connection 0
root@Saint-James:~/Lab-4.1-Web-Server#
```

Figure 35: Traversal attempt with percent-encoded dots (%2e%2e): 404 Not Found.

```

^L[2026-10-07 17:49:03.021] SECURITY: blocked path traversal attempt: '../.../etc/passwd' -> '/etc/passwd'
[2026-10-07 17:49:03.021] #41 127.0.0.1:43650 "GET ../.../etc/passwd HTTP/1.1" -> 404 Not Found (141 bytes, text/html; charset=utf-8) [worker :
]
| GET ../.../etc/passwd HTTP/1.1
| Host: localhost:8080
| User-Agent: curl/7.81.0
| Accept: */*
[2026-10-07 17:50:08.397] SECURITY: blocked path traversal attempt: '../.../etc/passwd' -> '/etc/passwd'
[2026-10-07 17:50:08.397] #42 127.0.0.1:52156 "GET ../%2e%2e/%2e%2e/etc/passwd HTTP/1.1" -> 404 Not Found (141 bytes, text/html; charset=utf-8) [
worker 2]
| GET ../%2e%2e/%2e%2e/etc/passwd HTTP/1.1
| Host: localhost:8080
| User-Agent: curl/7.81.0
| Accept: */*

```

Figure 36: The server log records each blocked attempt and where it would have led.

curl needs `-path-as-is` for this test. By default it removes `../` segments from the URL before sending, so without the flag the attack never reaches the server and the test proves nothing.

The server answers 404 rather than 403, so the response does not reveal whether the target exists. The script `scripts/test_http.sh` additionally tests encoded slashes (`..\%2f`), a path that starts inside the root and then climbs out, a NUL byte and a malformed escape. All seven cases passed: the five traversal attempts returned 404 and the two malformed URLs returned 400.

6.4 Why this matters

A traversal flaw lets anyone who can reach the server read any file the server process can read: `/etc/passwd`, application source code, configuration files, database passwords, API keys and private keys. Leaked credentials usually turn an information leak into a full compromise. The attack needs no authentication and nothing more than a URL, so it is easy to automate and is scanned for constantly.

Path traversal is CWE-22 and belongs to **A01: Broken Access Control**, the first category of the OWASP Top 10 (2021): the server fails to enforce the rule that clients may only read files inside the web root. The general lessons are to validate the outcome rather than the input text, to normalise data before checking it, and to run the server with as few file permissions as possible so that a mistake exposes less.

7 Conclusion

Design	Strength	Weakness
Single-threaded	Simplest; no shared state	One slow client blocks all others
Thread per client	Clients fully concurrent	Thread count set by the clients
Thread pool	Bounded resources	Clients wait when the pool is busy

The measurements match the designs. With 500 ms of work per client, the single-threaded server needed 5.02 s for ten clients because every client waited for all the ones ahead of it. One thread per client brought that down to 0.55 s, and the pool of ten served twenty clients in 1.03 s while using 11 threads where the thread-per-client server needed 21.

For a real server I would choose the thread pool. It gives nearly all of the concurrency of one thread per client, but its resource use is decided by the server rather than by whoever connects to it, and under overload it slows down instead of failing. The pool size and the queue then become the two settings to tune for the workload.

The security exercise showed that the networking code was the easy part. The original traversal check looked reasonable and was still bypassed by encoding two characters. Checking where a request actually ends up, rather than what its text looks like, is the approach I would reuse elsewhere.